

THE LADDER

A test rig for 64-bit mixers that does not saturate.

How to tell good mixing functions apart when every ordinary quick test says the same thing about all of them. A measuring instrument with built-in distrust of itself — fully reproducible.

Instruments that do not question themselves lie.

0x44 Zero Systems

Author: 0x44 DANTE

As of: 30 August 2026 (working draft)

What this is about

A **mixer** (hash finalizer) turns a structured number into a well-shuffled value. Structured means a customer ID, a timestamp, a counter. It sits in hash tables, in sharding, in random generators. Anyone building or choosing one needs an answer to the question: **is this one good?**

The problem: the usual tests saturate.

Avalanche tests measure whether flipping one input bit flips about half the output bits. That sounds reasonable, but practically every serious mixer scores near-perfect there, the weak ones included. An example from this rig: a chain with avalanche **0.4998** (ideal: 0.5) failed on the ladder at the very first checkpoint, after 1–2 kilobytes. The quick test certified perfection. What it could not see, by construction, was the structure behind it: every output byte depended on only two input bytes.

A test where everyone gets top marks does not separate. For a ranking you need a quantity that keeps growing with better quality instead of hitting its ceiling. That is what the ladder does.

The measuring principle

The ladder feeds the mixer the hardest realistic stream: counting numbers — exactly what counters, IDs and timestamps deliver in production. So that no mixer can tune itself to a single stream shape, the counter arrives in **disguises** (Evensen's RRC scheme): read backwards (bit reversal), turned into its opposite (complement), rotated by every position. 4 transformations $\times$ rotations give 32 to 256 streams depending on the stage.

The mixed stream is read by **PractRand**, an established test battery from random-number research, which reports how much data it needs before it can demonstrate a difference from real randomness.

That amount of data is the measurement. A weak mixer is exposed after kilobytes, a strong one holds terabytes. The scale does not saturate — it still separates where every quick test has long since handed out top marks:

Mixer	Cost	Dies on the ladder
Stafford mix13 (splitmix64)	12	2^{16} to 2^{21} (median 2^{19}), all 32 entry streams
Murmur3 finalizer (fmix64)	12	2^{14} to 2^{18} (median 2^{16}), all 32 entry streams
Moremur (Evensen)	12	16 of 32 disguises, from 2^{16}
mulfold $\times 2$ (bare)	8	11 of 32 disguises
one AES round, folded	~ 5	immediately, at 1–2 KB (2^{10})
lu9 (wyhash pattern)	9	holds 32/32 and 2^{34} — tears in the diploma at 2^{38}
meer10 (fold-xsh2-fold)	10	holds the diploma: 256/256 streams to 2^{40}
NASAM (Evensen)	~ 15	holds: passes RRC-64-42
mx3 rev2 (Kågström)	~ 17	holds: passes RRC-64-42

Verifiability rests on this: the verdict depends only on the byte stream, not on the machine. Anyone pushing the same mixer through the same ladder gets the same result; unlike runtime measurements, there is no noise to argue about here.

The stages — cheap kills first

Terabyte tests for every candidate would be waste: most die in seconds. The ladder is therefore a cascade — each stage costs a multiple of the previous one, and only those who pass climb.

Stage	Scope	Purpose
Gauntlet	4 streams × 64–256 MB	the four historically deadliest disguises first — kills the bulk in seconds
Smoke	32 streams × 2 GB	the entry test: 4 transformations x 8 of 64 rotations, none may fall
Depth test	64 streams × 16 GB (2^{34})	separates real candidates from shallow ones — some pass the smoke and die here
Diploma (RRC-64-40)	256 streams × 1 TB — ten streams at once on 8 physical cores	the test the literature respects; only for finalists

That the stages are not a ritual was shown twice. On 19 August an entire enumerated family of 23 chains, all of them past the full 32-stream smoke, died at depth. Not one survivor. On 20–21 August the cheapest passer (lu9) fell in the diploma at 2^{38} , after holding 2^{34} on 64 of 64 streams. **Passing a stage is a ticket to the next one, not a seal of quality**, and that holds right up to the last.

The anchors — why the rig may be believed

The property that decides whether a test bench is worth anything is its ability to **reject**. Measuring is the easy part. A test that never fails anyone looks exactly like a test that works, only cheaper.

Every run of this rig therefore begins with anchors, and there are six, in three pairs. Each pair asks its question from both sides, because a check that has only ever been seen to pass has not been seen to work:

Verdict	Anchor	What it proves
FAIL	a known weak mixer (mix13) MUST fall	the rig can convict
clean	a known strong one (NASAM) MUST hold	the rig does not convict everyone
NOT MEASURED	an unknown mixer name MUST come back unmeasured	the rig notices when nothing was measured
ABORTED	a watchdog kill MUST come back aborted	the rig notices when it stopped early
(it bites)	a stream that writes 16 KB and then freezes MUST be cut off	the stall detector fires at all
(the margin)	a healthy stream MUST survive the stall detector	and does not bite the living

If any of the six misbehaves, nothing is measured. No result from this rig exists without a passing anchor in the same run.

The biting test is the youngest of them, and it is there because of what its absence cost. Until 30 August 2026 the stall detector asked whether a stream had produced a *first* checkpoint, and the stall it is named after does not happen before the first checkpoint. It happens at about 8 KB, four checkpoints in. So the detector never fired, for its entire existence, while streams that stalled paid two full watchdog timeouts each to produce nothing: 1,409 of the 92,091 streams the finished cost-5 campaign logged, counted afterwards over the published archive, and bunched into regions rather than spread. The only anchor it had was the margin, and a margin can only ever show that a detector spares the healthy. Both directions, or neither.

The two mixer weights hold their slots because somebody else measured them, not because we rate them: Evensen publishes mix13 dying between 2^{16} and 2^{22} , and NASAM passing RRC-64-42. Our own mfx9 held that upper slot until 29 August 2026 and behaved perfectly in it — which is what made it hard to see that “mfx9 holds” only ever said the rig still agreed with itself.

The parable of the silent exit.

Why so much distrust? Because exactly this failure happened. PractRand 0.94 has a function missing a return value. That is undefined behaviour, and under optimisation it makes the process end quietly after the first checkpoint. Everything looked “clean to 2^{10} ”: every mixer, including the worst. It only came out through the contradiction with the literature, since mix13 appeared to pass and that could not be. The bug has been known upstream since 2019 and is fixed in newer versions; this rig runs the documented patch and proves its ability to reject before every run anyway.

The parable of the silent judge.

The first parable left a gap that stood for eleven days, in a rig built on distrust, by someone who had just been bitten by exactly this. The silent exit was found on 17 August 2026, the gap it left behind on the 28th. The two mixer anchors check whether the rig *judges correctly*, and in doing so assume that something was judged at all. A mixer that has not yet fallen after two kilobytes violates no expectation. **Both anchors would have waved the silent exit through.**

The same mistake sits one level higher, in the layer that *reads* the verdict rather than the one that writes it: ten places asked “is there a FAIL?” and treated the absence of one as a pass. That is only equivalent if the verdict space has two values. It has four, and two of them carry no FAIL:

Verdict	no FAIL?	actually clean?
clean to 2 ⁴⁰	yes	yes
FAIL at 2 ³⁸	no	no
ABORTED (timeout)	yes	no
NOT MEASURED	yes	no

The bottom two rows were the bug, in all ten places. One counted the clean streams by subtraction (total minus fallen minus aborted), so an unmeasured stream fell into “clean” by arithmetic alone. The diploma checked that every stream was booked and none had failed, but never that any of them reached the target depth: “clean to 2³³” from a run that died early would have read as a pass over the full terabyte.

All of it is now one rule that every decision has to ask, rather than a habit applied by hand at each site:

```
def is_clean(v):
    # Absence of a conviction is not innocence --
    # ask for the acquittal in writing.
    return v.startswith("clean")
```

Did anything happen at all? The check after the run.

Anchors run before a measurement. Afterwards a second question can be asked of the log itself: the test program records every doubling of the data volume separately, along with the number of sub-tests performed. From that one can recompute:

- Is the chain of checkpoints gapless, or does it break off somewhere?
- Did a run counted as clean reach the full target depth — or does it end earlier (a *fallen* one may end early, a clean one may not)?
- Does the number of sub-tests grow monotonically with the data volume?
- Did the process terminate normally?

If any of those numbers disagrees with expectation, the run counts as **not performed** — neither passed nor fallen. That is the same rule as for a stall, one level deeper: a measurement that did not take place may not support a verdict.

This check too had to prove it can reject. It is tested against artificial failures: broken chains, gaps, runs ending too early, falling sub-test counts. Among them is the case that reproduces the historical bug. During the build, incidentally, exactly the trap it guards against occurred once more: one of the test cases was caught by the wrong rule, so the rule actually meant went untested. A watchdog with teeth nobody has ever tested is not a watchdog.

Where this check lives, and where it does not.

It belongs to the search rig, which is not published. The ladder in this repository does **not** have it: it tests the target depth only in the diploma tally and the checkpoint chain only as far as the stall detector needs it, and for the monotonicity of the sub-test counts it has nothing at all. This paragraph would otherwise read as though the check were aboard everywhere. It is an open gap in the public instrument, named in its README and named here, because the rig this paper describes has to be the rig the reader can run.

An unmeasurable stream is never clean.

If a test pair stalls (which happens under load), a watchdog aborts it and marks the stream ABORTED — it is redone or reported as unclear, but never counted as passed. An invented number is worse than no number.

What the ladder is, and what it is NOT

It is a judge, not a sculptor.

The ladder says WHETHER a chain passes — it gives no gradient along which better constants could be shaped from scratch. The attempt to breed our own constants with the ladder alone failed empirically (18 August, zero survivors): below passing, its answer is too coarse to point a search direction. Anyone wanting to shape needs a finer metric as well; the ladder stays the last word.

It measures streams, not speed.

Whether a mixer is fast is answered by a cost model or a runtime measurement — both separate instruments. The ladder answers only the quality question.

Passing is not a proof.

“Holds 2 GB on 32 streams” means exactly that — no more. The next stage can kill the candidate (and regularly does). Conversely, failing is final: a demonstrated difference from randomness does not go away again.

Novelty is a literature question.

That a chain passes says nothing about whether it has been published somewhere already. Before any publication comes the literature check — the ladder does not replace it.

Reproducibility

The rig consists of three parts, all documented openly: a feeder that produces the disguises (C++, one file), a driver that runs test pairs in parallel and collects verdicts (Python), and an unmodified PractRand 0.94 with two documented build traps: it needs the GNU dialect of the C++14 standard (otherwise input stays in text mode under Windows and line endings destroy the stream silently) and the return-value patch mentioned above. Byte streams run over native pipes; any intermediate layer that translates line endings makes the measurement worthless.

Anyone wanting to re-measure needs no access to this machine: mixer, disguise and test program determine the result completely.

Related rigs

This ladder does not stand alone. Pelle Evensen designed the test scheme and used it to establish NASAM. Martin Leitner-Ankerl measured mixers over RRC streams (no complement) against PractRand 0.95 and plotted their speed against test depth, with a Pareto front — twenty listed, fifteen with results (github.com/martinus/better-faster-stronger-mixer, README January 2020, counted 30 August 2026). That is the closest related rig, and the only one known to me that connects both axes. Jon Maiga tabulated mixer skeletons against PractRand depth. The measuring principle is Evensen's. What this rig adds is the mode of operation: a cascade with cost economy, anchors as the starting condition of every run, and the ability to enumerate whole spaces instead of selecting candidates.

In brief

The question	How do you tell good 64-bit mixers apart when quick tests say the same thing about all of them?
The principle	Counter streams in disguises (RRC) through the mixer into PractRand; the amount of data up to the demonstrated difference is the measurement — it does not saturate.
The cascade	Gauntlet (MB) → smoke (32×2 GB) → depth (64×2 ³⁴) → diploma (256×1 TB). Cheap kills first.
The anchors	Six before every run, in three pairs, each pair asked from both sides: known-weak MUST fall and known-strong MUST hold; an unmeasurable stream MUST come back unmeasured and a killed one MUST come back aborted; a frozen stream MUST be cut off and a healthy one MUST survive the cut. Stalls never count as clean.
The limits	A judge, not a sculptor. It measures quality and says nothing about speed. Passing buys the next stage and nothing more. Novelty is settled by the literature.

Instruments that do not question themselves lie.

0x44 Zero Systems

Author: 0x44 DANTE

As of: 30 August 2026 (working draft). Results obtained with this rig are in the paper *The Cost Limit*.